

合肥工业大学

智能无人飞行器设计

设计报告

学 院	计算机与信息学院
专 业 班 级	物联网 23 级-1 班
学生姓名及学号	莫仁鹰 2023212167
指导教师	【指导教师姓名】
课题名称	基于 RK3566 的智能无人飞行器 视觉识别系统设计
2026 年 6 月 17 日	2026 年 6 月 17 日

一、课题概述

随着无人机技术的飞速发展，智能无人飞行器在航拍测绘、农业植保、物流配送、灾害搜救等领域得到了广泛应用。近年来，基于深度学习的目标检测技术（如 YOLO 系列算法）与嵌入式计算平台（如瑞芯微 RK3566）的结合，使得无人飞行器具备了实时视觉识别与自主决策能力，推动了智能无人系统向轻量化、低功耗、高实时性方向发展。

本课题以 Cessna 182 固定翼航模为载体，围绕“嵌入式视觉识别系统”这一核心任务展开设计与实验。主要工作包括：飞机机体组装与调试、Pixhawk 飞控固件升级与参数配置、RK3566（立创泰山派）开发板系统镜像烧录与驱动部署、YOLO 目标检测数据集构建与标注、YOLOv5 模型训练与 ONNX 格式导出、以及 RKNN 模型转换与边缘端部署。

本项目的设计目标是构建一套完整的无人飞行器视觉识别系统原型：由 Pixhawk 飞控负责飞行姿态控制，RK3566 开发板承担机载视觉推理任务，通过摄像头采集图像并运行轻量化 YOLO 模型实现实时目标检测。整个系统从硬件组装、固件烧录、模型训练到边缘部署，覆盖了嵌入式 AI 系统开发的全流程，对培养系统级工程能力具有重要的实践意义。

本报告将按照课题任务、技术方案、设计实现与测试、课程总结四个部分，详细阐述设计过程中的技术路线、实验步骤、遇到的问题及解决方案，力求完整呈现本次课程设计的全貌。

二、课题任务

2.1 功能需求

本课题需要完成以下核心功能：

(1) 飞机机体组装：完成 Cessna 182 固定翼航模的机械结构组装，包括机身、机翼、尾翼、起落架、电机、舵机、螺旋桨等部件的安装与固定。

(2) 飞控系统配置：对 Pixhawk 飞控进行固件升级（ArduPlane 固件），完成遥控器校准、飞行模式设置、传感器标定等基本参数配置，使飞控具备基本的飞行控制能力。

(3) 嵌入式开发板部署：在 RK3566（泰山派）开发板上烧录 Ubuntu 系统镜像，配置开发环境，安装必要的驱动和依赖库。

(4) 视觉识别模型开发：使用 Labellmg 工具标注目标检测数据集，基于 YOLOv5 框架训练目标检测模型，将训练后的模型转换为 ONNX→RKNN 格式，部署到 RK3566 开发板上实现边缘端推理。

(5) 系统集成与测试：将飞控、开发板、摄像头等硬件集成到飞机机体内，进行联调测试。

2.2 技术指标

(1) 飞控固件版本：ArduPlane V4.x 稳定版，支持多飞行模式切换。

(2) 开发板操作系统：基于 Ubuntu 的 Linux 系统（Buildroot/Debian），支持 SSH 远程登录和桌面显示。

(3) 目标检测模型：YOLOv5s/YOLOv5n，输入分辨率 640×640，推理速度满足边缘端实时性要求。

(4) 模型格式：PyTorch (.pt) → ONNX (.onnx) → RKNN (.rknn)，最终在 RK3566 NPU 上加速推理。

(5) 摄像头接口：USB 摄像头或 MIPI CSI 摄像头模块，支持 720P 以上分辨率采集。

三、技术方案及关键问题

3.1 总体技术方案

本项目采用“飞控+视觉处理板”双板架构。Pixhawk 飞控负责飞行姿态控制和传感器数据融合，RK3566 开发板负责视觉图像采集与 AI 推理。两者之间通过串口（UART/TELEM）进行通信，实现视觉识别结果向飞控的反馈。整体技术路线如下：

硬件层面：Cessna 182 航模机体 → 电机/舵机/电调安装 → Pixhawk 飞控安装与接线 → RK3566 开发板安装 → 摄像头模块连接 → 电池/电源分配板布线。

软件层面：Pixhawk 固件烧录（Mission Planner）→ RK3566 系统镜像烧录（RKDevTool）→ 开发环境搭建（Python/RKNN-Toolkit2）→ 数据集标注（Make Sense）→ YOLOv5 模型训练 → ONNX 导出 → RKNN 转换 → 边缘端部署推理。

3.2 开发平台选型

（1）飞控平台：选用 Pixhawk 系列飞控，其基于 STM32 处理器，支持 ArduPilot 开源固件，生态成熟、社区活跃，是教学和科研领域最常用的飞控平台。地面站软件使用 Mission Planner（Windows 平台），支持固件升级、参数配置、实时数据监控和航线规划。

（2）视觉处理平台：选用瑞芯微 RK3566 芯片的立创泰山派开发板。RK3566 内置 0.8TOPS 算力的 NPU（神经网络处理单元），支持 RKNN 推理框架，能够高效运行轻量化深度学习模型。相比树莓派等通用开发板，RK3566 在 AI 推理场景下具有显著的性能和功耗优势。

（3）目标检测框架：选用 YOLOv5，其在检测精度和推理速度之间取得了良好平衡。YOLOv5 提供多种模型规格（n/s/m/l/x），适合在不同算力平台上部署。训练框架基于 PyTorch，导出支持 ONNX 格式，可通过 RKNN-Toolkit2 转换为 RK3566 NPU 可执行的 RKNN 格式。

3.3 关键问题分析

（1）USB 数据线识别问题：在飞控固件烧录过程中，最初使用的 Micro USB 线为充电线（仅含电源线，无数据线），导致电脑无法识别 Pixhawk 设备。该问题的根本原因是充电线只有 2 根芯（VCC+GND），而数据传输需要 4 根芯（VCC+GND+D++D-）。解决方案：购买专用 Micro USB 数据线后问题立即解决。

（2）RK3566 开发板驱动兼容性：在 Windows 11（Build 26200）环境下，泰山派通过 USB 连接电脑后，RNDIS 网络设备虽能被识别但无法建立网络链

路。经排查发现，微软在 Windows 11 新版本中移除了对 RNDIS 驱动的完整支持（MediaConnectionState 始终为 Disconnected）。该问题导致无法通过 USB 网络直接 SSH 登录开发板。

（3）实验器材管理问题：项目前期完成初步组装后将器材放置于实验室，后发现部分零件（包括完整的机身部件）被实验室其他人员取走更换。在验收前两天，指导教师联系厂商重新调配了完整部件，才得以继续进行后续实验。此问题提醒了在共享实验室环境中做好器材标识和保管的重要性。

（4）模型格式转换兼容性：ONNX 模型转换为 RKNN 格式时，需要注意算子兼容性。RKNN-Toolkit2 对部分 ONNX 算子支持有限，需在导出 ONNX 时设置合适的 opset 版本，并在转换配置中指定正确的目标平台（rk3566）和量化策略。

四、设计实现及测试

4.1 飞机机体组装

本实验使用的飞机模型为 Cessna 182 固定翼航模，机翼翼展约 1.2m，采用 EPO 材质机身。组装过程在合肥工业大学实验室内完成，由小组成员协作进行。主要组装步骤包括：

(1) 机身主体与机翼连接：将左右机翼通过碳纤维管插入机身预留的翼管孔内，使用胶水和扎带固定。机翼上预装有副翼舵机，需将舵机延长线穿过机翼内部引出至机身。

(2) 尾翼安装：将水平尾翼和垂直尾翼插入机身尾部卡槽，固定升降舵和方向舵舵机，连接舵机摇臂与控制面。

(3) 电机与螺旋桨安装：将无刷电机固定在机头电机座上，连接电调（ESC）的三相线，安装螺旋桨并确认旋转方向正确。

(4) 起落架安装：前起落架和主起落架分别固定在机身底部对应位置，确保飞机放置时姿态水平。

(5) 电子设备安装：将 Pixhawk 飞控通过减震座固定在机身内部中心位置，RK3566 开发板固定在飞控上方或旁边，摄像头模块安装在机头下方或驾驶舱位置。所有设备通过电源分配板统一供电。

组装过程中，小组成员分工合作：部分成员负责机械结构拼装，部分成员负责电子接线。下图展示了组装过程的实景照片。



图 4.1 小组成员在实验室内协作组装 Cessna 182 飞机

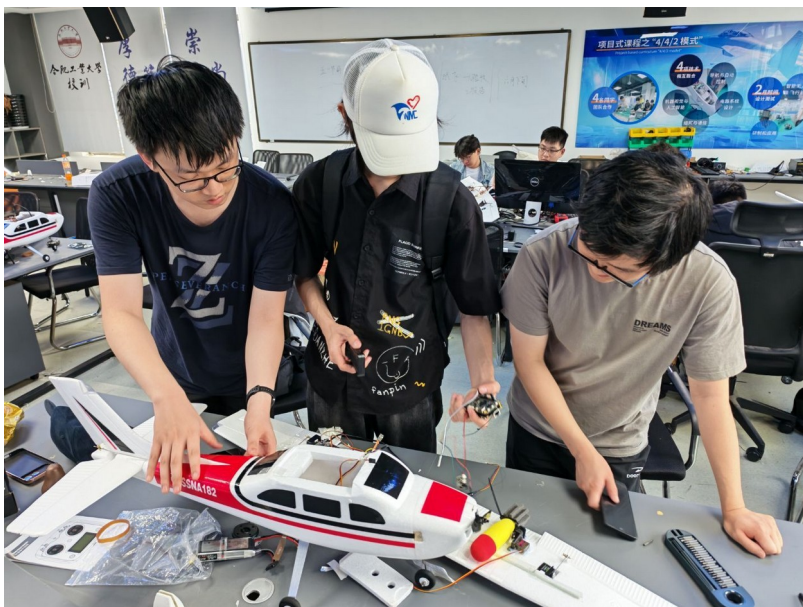


图 4.2 安装飞控与开发板接线



图 4.3 机身内部电子设备布局

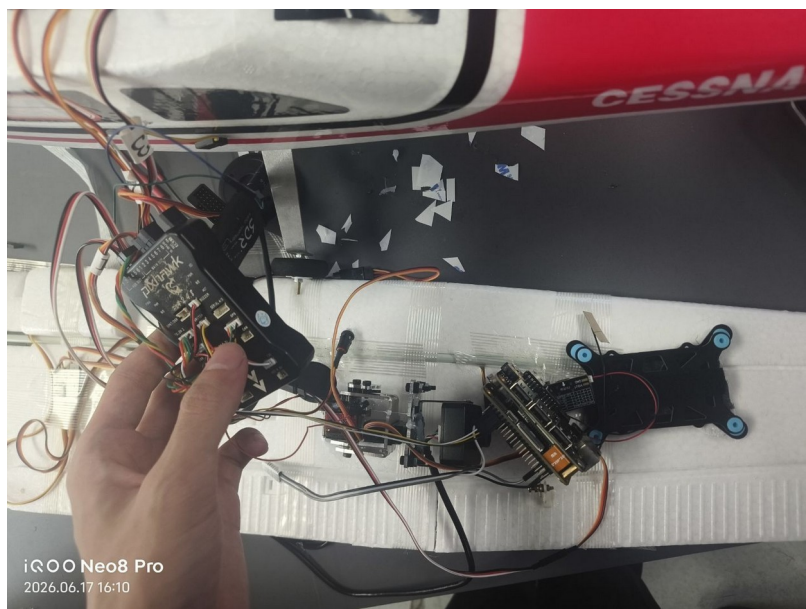


图 4.4 机身内部 Pixhawk 飞控与减震座安装 (近景)

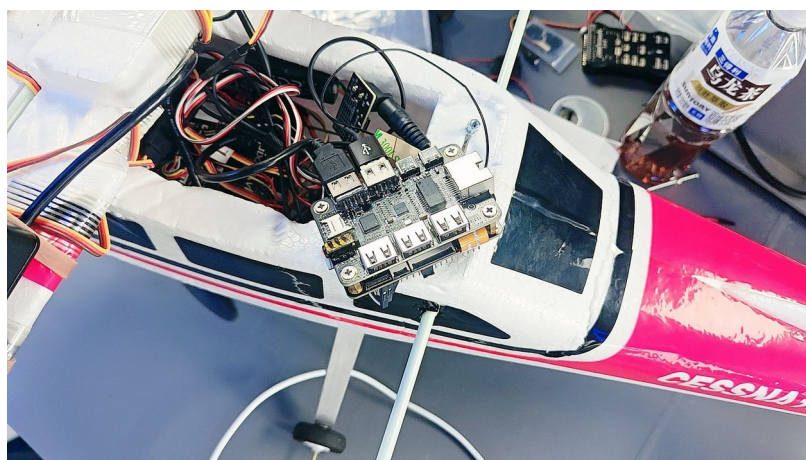


图 4.5 RK3566 泰山派开发板安装在飞机机舱内

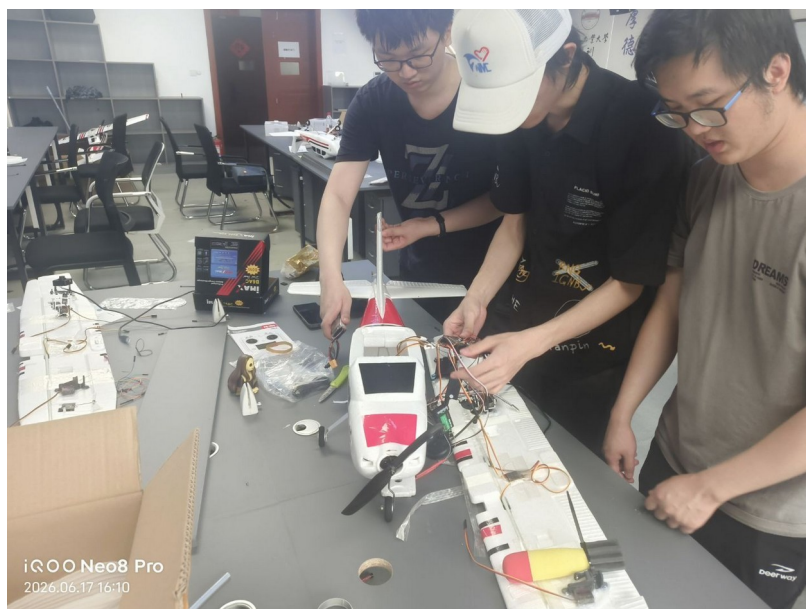


图 4.6 小组成员进行飞机接线与调试

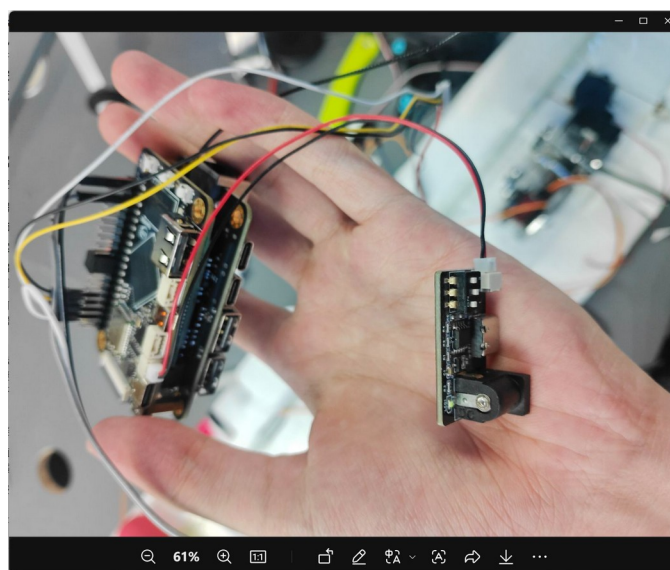


图 4.7 RK3566 开发板与摄像头模块连接

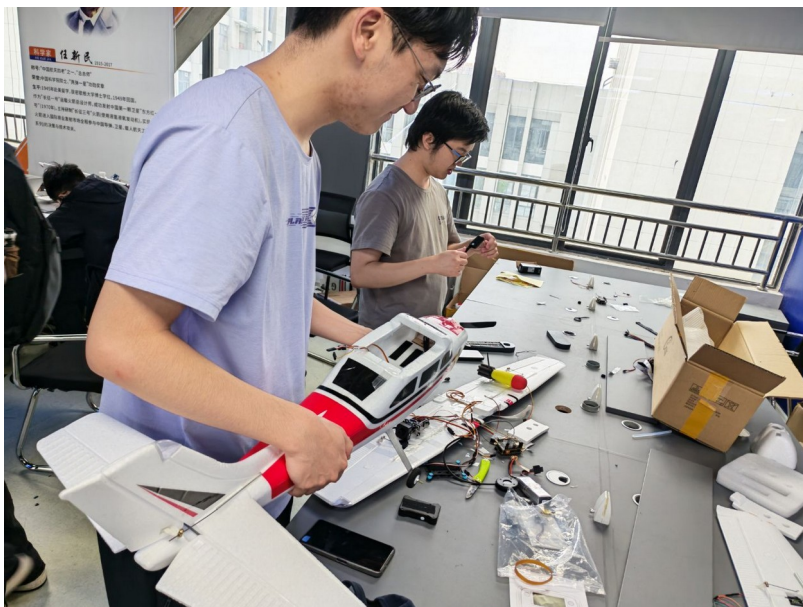


图 4.8 飞机内部走线布局



图 4.9 机翼与机身对接固定



图 4.10 实验室全景——各小组同时进行飞机组装

在组装过程中遇到了一个较为棘手的问题：项目初期完成初步组装后将飞机放置在实验室，后发现部分零件被其他实验室成员取走替换。在验收前两天，指导教师联系到厂商重新调配了完整的机身部件，小组成员紧急完成了重新组装工作。这一经历提醒我们，在共享实验室环境中需要做好器材的标识与保管。

4.2 飞控固件升级及参数烧录

本节对应实验 3.2 的内容。Pixhawk 飞控需要烧录 ArduPlane 固件才能控制固定翼飞机。本实验使用 Mission Planner 地面站软件完成固件升级和参数配置。

4.2.1 实验环境

硬件：Pixhawk 2.4.8 飞控、Micro USB 数据线、Windows 电脑。

软件：Mission Planner 地面站（V1.3.x），已预装在课程资源包中。

4.2.2 固件烧录步骤

（1）使用 Micro USB 数据线将 Pixhawk 连接至电脑。首次连接时 Windows 会自动安装驱动，设备管理器中应出现对应的 COM 端口。

（2）打开 Mission Planner，点击右上角“初始设置”→“安装固件”，选择 ArduPlane（固定翼）图标，等待固件下载完成后自动烧录。烧录过程中 Pixhawk 指示灯会快速闪烁，完成后提示“Upload Done”。

（3）烧录完成后断开 USB 重新连接，在 Mission Planner 左上角选择正确的 COM 端口和波特率（115200），点击“连接”按钮与飞控建立通信。

4.2.3 遇到的问题及解决

在本实验过程中遇到了一个关键问题：使用最初配备的 Micro USB 线连接 Pixhawk 后，飞控指示灯亮起（说明供电正常），但电脑的设备管理器中始终无法识别到 COM 端口，Mission Planner 也无法连接。

经过排查分析，确定问题出在 USB 数据线上。初始使用的线缆实际上是充电线，内部仅有 VCC 和 GND 两根电源线，缺少 D+ 和 D- 两根数据线，因此无法进行 USB 数据通信。这是一个非常常见但容易被忽视的问题——外观上充电线和数据线几乎无法区分。

解决方案：在网上购买了专用的 Micro USB 数据线（确认支持数据传输），更换后设备管理器立即识别到了 COM 端口，Mission Planner 也成功连接并完成了固件烧录。

【占位符 A：Mission Planner 固件烧录界面截图（包括：选择 ArduPlane 固件、烧录进度、Upload Done 提示）——需要在本地电脑上连接 Pixhawk 后截图，截图中需包含学号 2023212167 水印】

【占位符 B：Mission Planner 连接飞控成功界面（包括：COM 端口选择、连接成功后的 HUD 仪表盘显示）——需要在本地电脑截图】

【占位符 C：设备管理器中 Pixhawk COM 端口识别截图——需要在本地电脑截图】

4.2.4 参数配置

固件烧录完成并成功连接后，需要进行基本参数配置：

(1) 加速度计校准：按照 Mission Planner 提示，依次将飞控放置为水平、左侧、右侧、机头朝下、机头朝上、倒置六个姿态，完成六面校准。

(2) 遥控器校准：打开遥控器，在 Mission Planner“初始设置”→“遥控器校准”中，拨动所有摇杆和开关至极限位置，记录各通道的 PWM 范围。

(3) 飞行模式设置：在“飞行模式”页面设置至少三种飞行模式（MANUAL 手动、STABILIZE 增稳、FBWA 辅助飞行），分配到遥控器的三段开关通道。

【占位符 D : Mission Planner 加速度计校准/遥控器校准/飞行模式设置界面截图——需要在本地电脑截图】

4.3 RK3566 开发板系统镜像烧录

本节对应实验 3.3 的内容。RK3566 泰山派开发板需要烧录 Linux 系统镜像才能正常使用。本实验使用瑞芯微官方的 RKDevTool 工具在 Windows 上通过 USB 完成镜像烧录。

4.3.1 实验环境

硬件：立创泰山派 RK3566 开发板、Type-C 数据线（用于烧录）、Type-C 电源线（5V/2A）、HDMI 显示屏。

软件：RKDevTool（瑞芯微开发工具 V2.96），DriverAssistant（USB 驱动安装工具），系统镜像文件（update.img）。

4.3.2 烧录步骤

（1）安装 USB 驱动：运行 DriverAssistant 中的 DriverInstall.exe，点击“驱动安装”，等待提示安装成功。该驱动使 Windows 能识别 RK3566 处于 Loader 模式或 Maskrom 模式的 USB 设备。

（2）进入 Loader 模式：按住泰山派上的 RECOVERY 按键不放，同时将 Type-C 数据线连接至电脑 USB 口（此时 Type-C 口兼做数据和供电）。RKDevTool 底部状态栏应显示“发现一个 LOADER 设备”。

（3）加载镜像文件：在 RKDevTool 中点击“升级固件”→“固件”按钮，选择系统镜像文件 update.img。

（4）执行烧录：点击“升级”按钮，工具开始将镜像写入开发板的 eMMC 存储。烧录过程约 3-5 分钟，期间进度条会显示写入进度。完成后提示“升级成功”，开发板自动重启。

4.3.3 烧录结果验证

烧录完成后，使用独立的 Type-C 电源线为开发板供电（不通过电脑 USB 供电，避免电流不足），通过 HDMI 线连接显示屏。开发板正常启动后显示 Linux 登录界面，用户名为“lckfb”。下图为实际拍摄的启动界面：

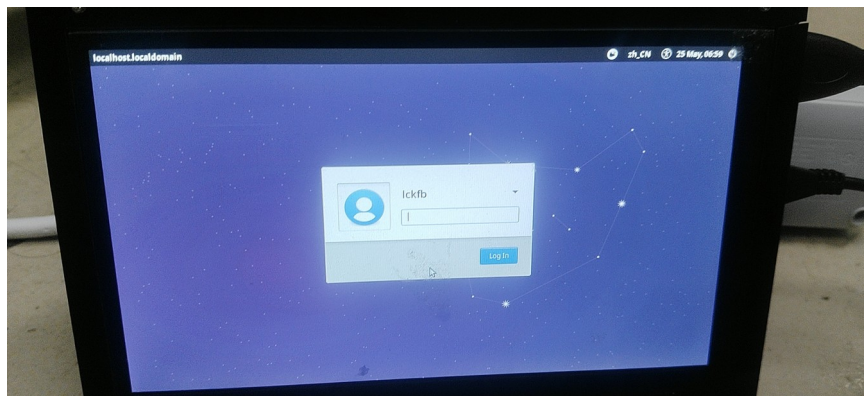


图 4.11 RK3566 泰山派开发板启动后的 Linux 登录界面

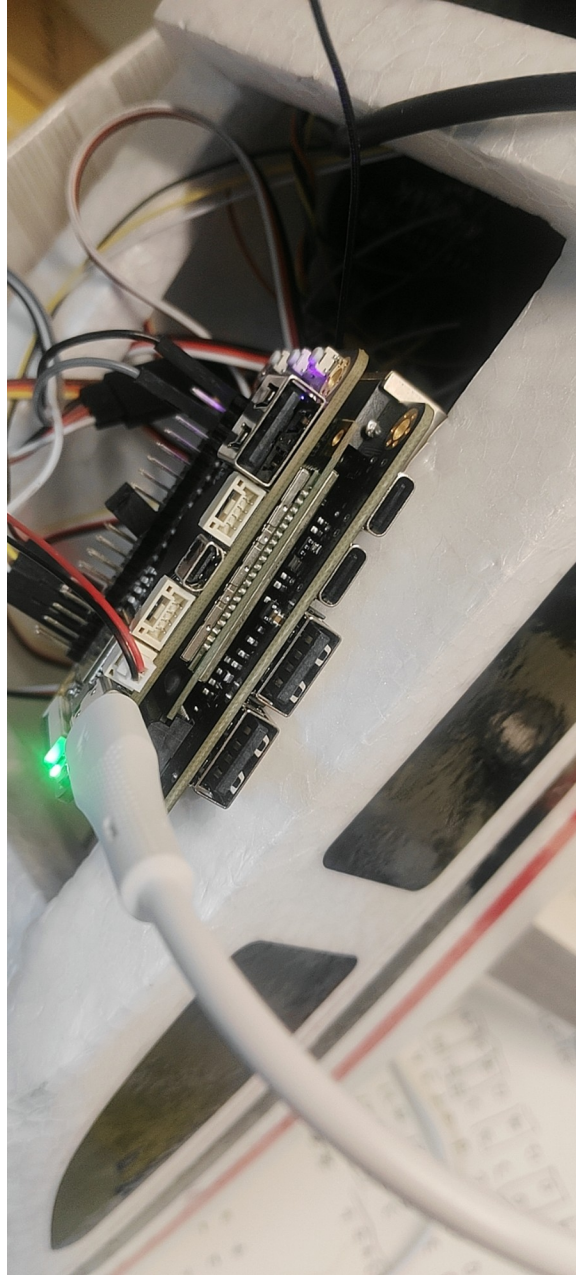


图 4.12 泰山派开发板上电状态 (绿色 LED 亮起, Type-C 供电与 HDMI 连接)

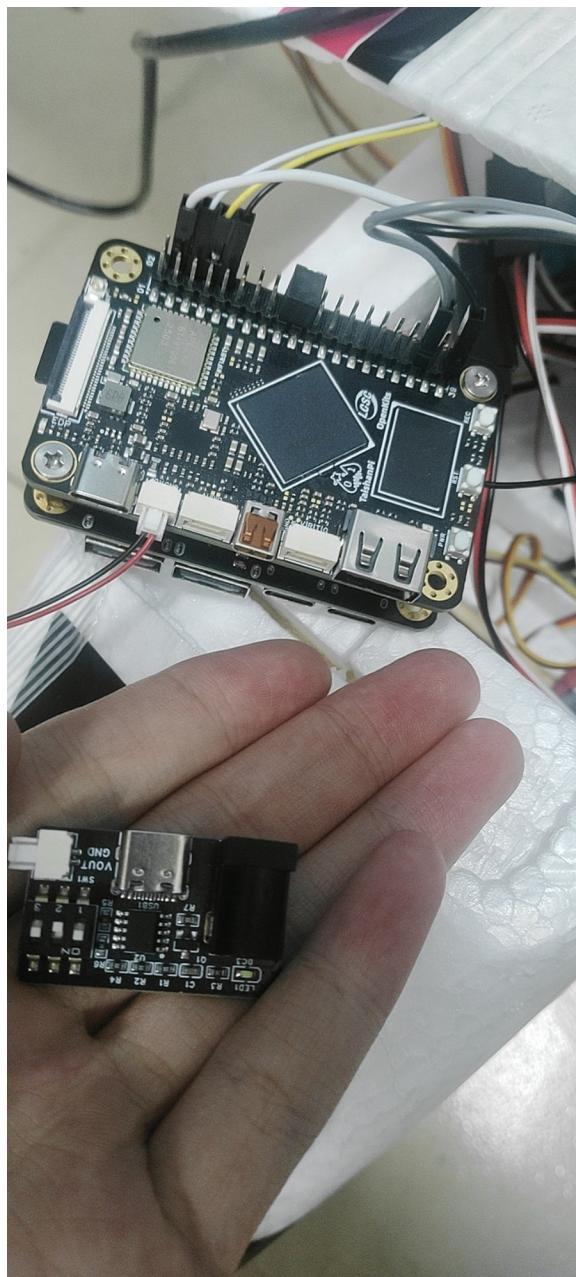


图 4.13 RK3566 泰山派开发板与电源模块近景

4.3.4 遇到的问题及解决

(1) HDMI 无信号问题：首次连接 HDMI 显示屏后显示“No Signal”。经排查发现开发板未上电（LED 指示灯未亮）。原因是仅通过电脑 USB 口供电，电流不足（电脑 USB 口一般仅提供 500mA）。解决方案：使用独立的 Type-C 充电器（5V/2A）为开发板供电，开发板正常启动。

(2) USB RNDIS 网络连接失败：希望通过 USB 网络方式从电脑 SSH 登录开发板，设备管理器中能看到“Remote NDIS based Internet Sharing

Device”（VID_0525&PID_A4A2），但网络适配器状态始终为“已断开连接”（MediaConnectionState=Disconnected，LinkSpeed=0 bps）。经查明，Windows 11 Build 26200 版本已移除对 RNDIS USB 网络的完整支持，该问题属于操作系统兼容性限制，非硬件故障。

（3）ADB 连接失败：尝试使用 adb devices 命令检测开发板，结果为空列表。原因是泰山派出所厂镜像默认未启用 ADB 调试模式。该问题需要在开发板端的系统设置中手动开启 USB 调试。

【占位符 E：RKDevTool 烧录过程截图（包括：识别 Loader 设备、加载固件、烧录进度、升级成功提示）——需要在本地电脑连接泰山派后截图】

4.4 飞机嵌入式系统硬件设计

本节对应实验 5.1 的内容。飞机嵌入式系统的硬件设计主要涉及各电子模块之间的接线与集成。

4.4.1 系统硬件架构

整体硬件架构如下：

- 飞控模块：Pixhawk 2.4.8，负责姿态解算、航线控制，通过 PWM 信号控制电机和舵机。
- 视觉处理模块：RK3566 泰山派开发板，通过 USB 连接摄像头，运行 RKNN 推理程序。
- 电源模块：锂电池（3S/4S）→ 电源分配板 → 分别为飞控（5V BEC）、开发板（5V Type-C）、电机（直连电调）供电。
- 通信模块：飞控与开发板之间通过 UART 串口通信（TELEM2 端口），传输视觉识别结果。
- 外设模块：GPS 模块（连接飞控 I2C/UART 口）、数传模块（连接 TELEM1 口用于地面站通信）、遥控接收机（连接 RC IN 口）。

4.4.2 接线方案

Pixhawk 飞控的主要接线包括：

- (1) MAIN OUT 1-4 通道分别连接副翼×2、升降舵、方向舵的舵机信号线。
- (2) MAIN OUT 3 通道连接电调的油门信号线（或使用 AUX 通道）。
- (3) POWER 端口连接电源模块，同时实现供电和电池电压/电流监测。
- (4) RC IN 端口连接遥控接收机的 SBUS/PPM 信号线。
- (5) TELEM2 端口通过杜邦线连接 RK3566 开发板的 UART 口。

RK3566 开发板的接线包括：

- (1) Type-C 供电口连接 5V 降压模块输出。
- (2) USB 口连接摄像头模块。
- (3) UART 口连接 Pixhawk TELEM2（TX-RX 交叉连接）。

注意：由于本学期实验时间限制，焊接电路板部分未能完成。实际接线采用杜邦线和排线方式实现各模块间的电气连接，虽然不如焊接稳固，但在实验室测试环境下可以满足功能验证需求。

【占位符 F：飞机硬件接线示意图/实物接线照片（包括：Pixhawk 接线全貌、电源分配板、开发板与飞控连接方式）——如有实验室现场照片可补充】

4.5 图片数据标注与 YOLO 格式数据集构建

本节对应实验 6.1 的内容。目标检测模型的训练需要高质量的标注数据集，本实验使用在线标注工具 Make Sense (makesense.ai) 对采集的图片进行标注，并将标注结果导出为 YOLOv5 所需的 YOLO 格式。

4.5.1 数据采集

训练数据来源于课程提供的坦克目标数据集，包含不同场景、不同角度下的坦克目标航拍图片。图片以 JPG 格式存储，分辨率不一。最终数据集包含训练集 613 张图片和验证集 218 张图片，按约 3:1 的比例划分。

4.5.2 Make Sense 在线标注工具使用

Make Sense (<https://www.makesense.ai/>) 是一款基于浏览器的免费在线图片标注工具，无需安装任何软件，直接在浏览器中即可完成标注工作。其操作流程如下：

(1) 打开 Make Sense 官网，点击“Get Started”进入标注界面。首页展示了工具的主要功能特性，包括免费使用、隐私保护（图片不上传服务器）、支持多种标注格式等。

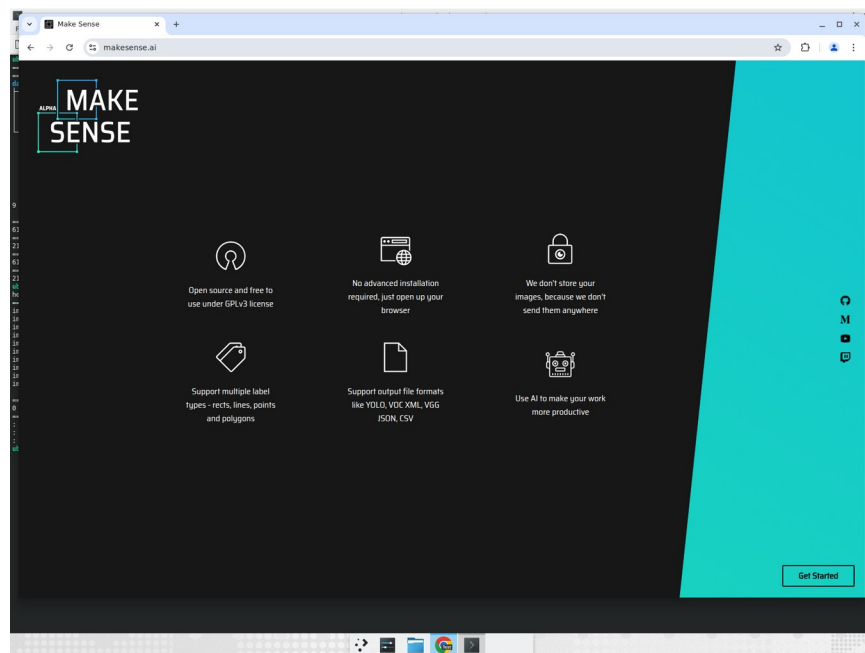


图 4.14 Make Sense.ai 首页界面

(2) 上传待标注图片：点击“Drop images or Click here to select”区域，批量选择需要标注的图片文件上传。上传完成后界面显示已加载的图片数量。

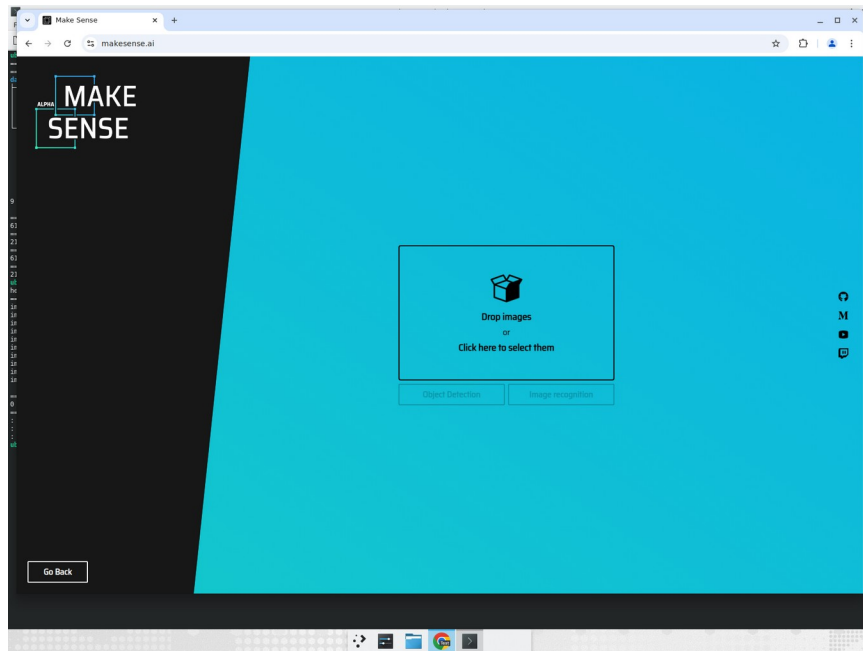


图 4.15 Make Sense 图片上传界面

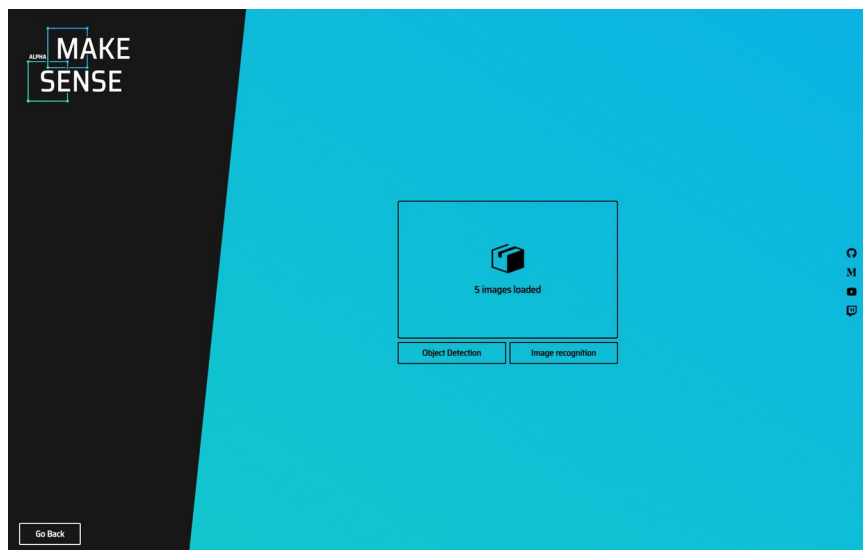


图 4.16 确认图片加载成功（显示已加载图片数量）

(3) 选择标注模式：点击“Object Detection”进入目标检测标注模式。该模式支持使用矩形边界框对图片中的目标进行标注。

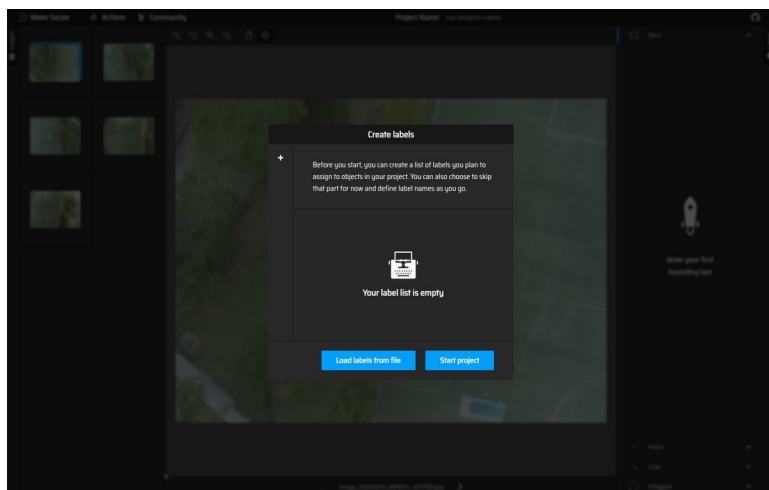


图 4.17 选择 Object Detection 标注模式

(4) 创建标签类别：在弹出的标签管理对话框中输入类别名称（本实验中为“tank”），点击添加创建标签。标签创建后即可开始标注工作。

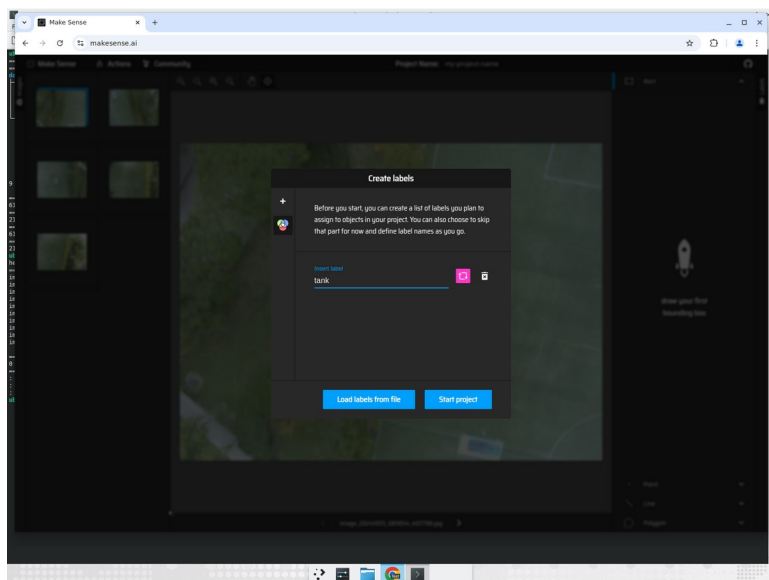


图 4.18 创建“tank”标签类别

(5) 进入标注工作区：左侧为缩略图列表，可快速切换待标注图片；中央为主画布区域，用于绘制边界框；右侧为标签列表和标注信息面板。

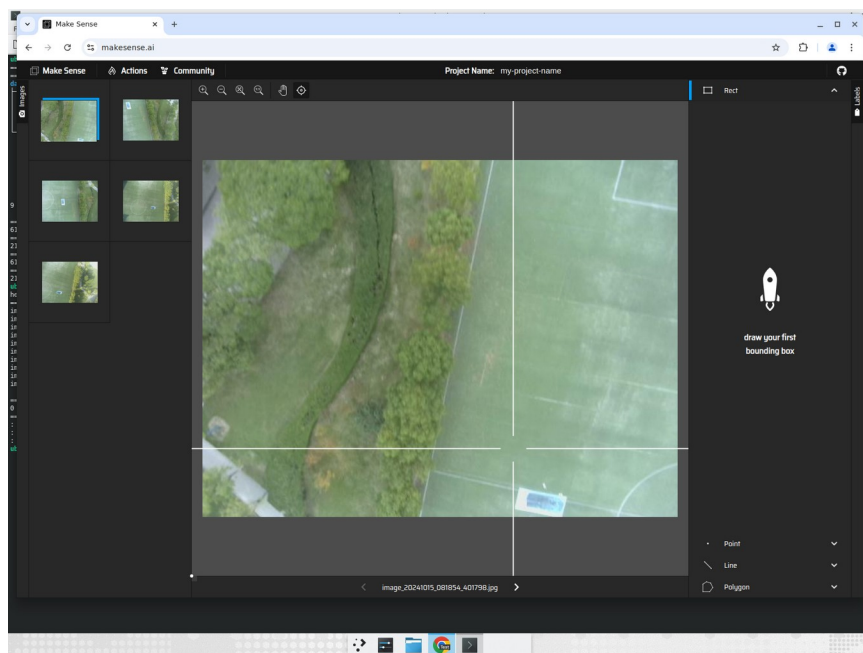


图 4.19 Make Sense 标注工作区界面

(6) 绘制边界框：在主画布上按住鼠标左键拖动，在目标周围绘制矩形边界框，释放鼠标后选择对应的类别标签（tank）。标注完成的边界框以粉色高亮显示，右侧面板同步显示当前图片的所有标注信息。

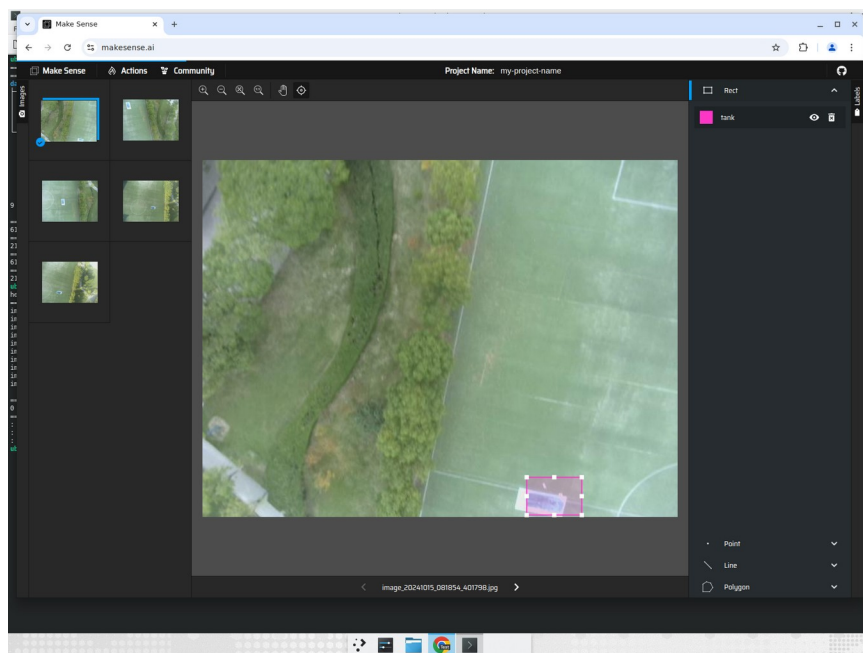


图 4.20 在航拍图片上标注坦克目标（粉色边界框标记为 tank）

(7) 导出 YOLO 格式标注：标注完成后，点击“Actions”→“Export Annotations”，选择“YOLO”格式导出。导出的标注文件为.txt 格式，每行包

含：类别编号、中心 x 坐标、中心 y 坐标、宽度、高度（均归一化到 0-1 范围），符合 YOLOv5 训练所需的数据格式。

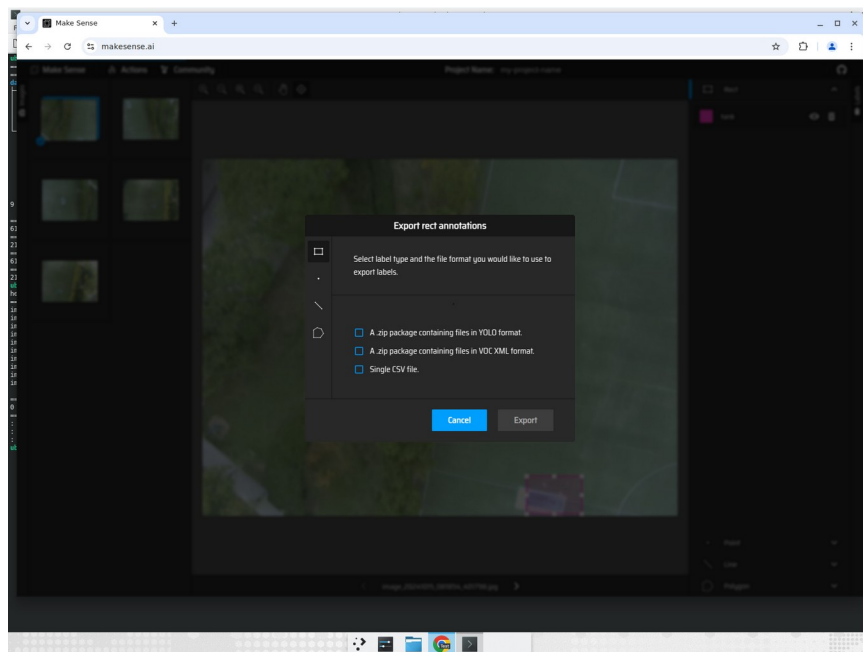


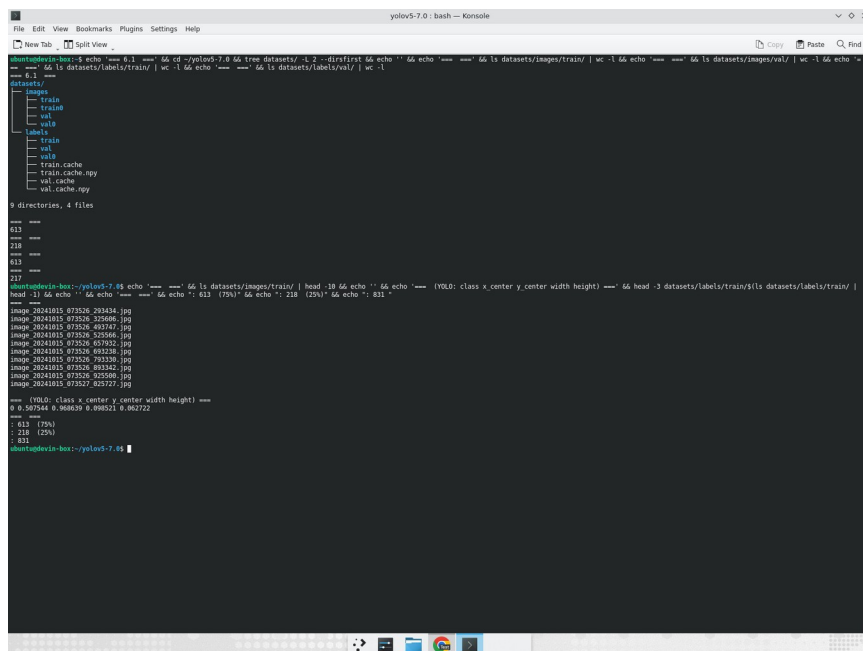
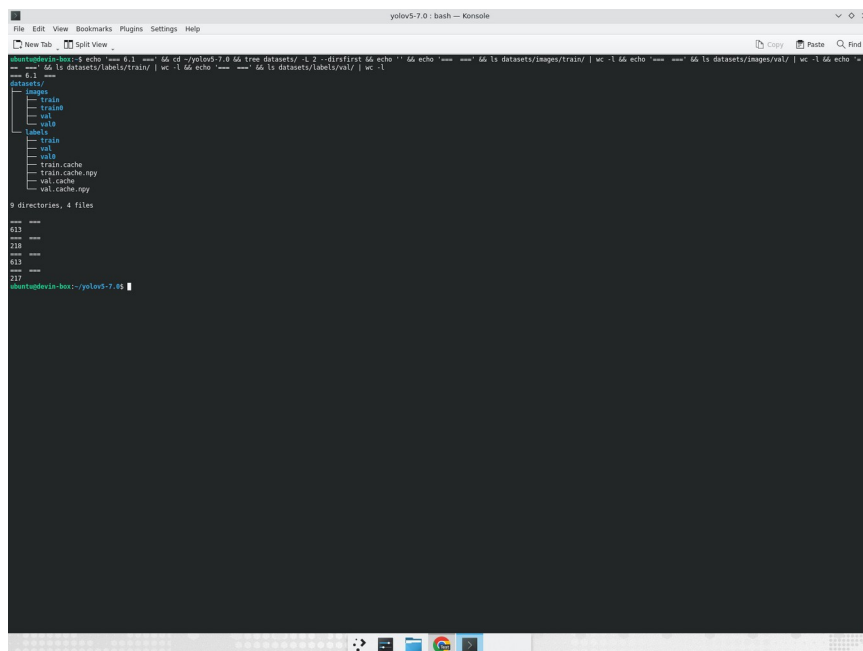
图 4.21 导出 YOLO 格式标注文件

4.5.3 数据集目录结构

YOLOv5 要求数据集按以下目录结构组织：

```
dataset/  
├── images/  
│   ├── train/      # 训练集图片 (613 张)  
│   └── val/        # 验证集图片 (218 张)  
├── labels/  
│   ├── train/      # 训练集标注 (613 个 .txt 文件)  
│   └── val/        # 验证集标注 (218 个 .txt 文件)  
└── cczz.yaml       # 数据集配置文件
```

在云服务器上验证数据集目录结构，确认训练集和验证集图片数量与标注文件一一对应。下图展示了实际数据集的终端目录树结构和文件统计信息：



`data.yaml` 配置文件指定了训练集、验证集路径和类别名称列表（`nc: 1, names: ['tank']`），是 YOLOv5 训练启动的入口配置。

4.6 训练环境搭建与模型训练

本节对应实验 6.2 的内容。在标注好数据集后，需要在云服务器上搭建 YOLOv5 训练环境并进行模型训练。本实验使用 Ubuntu 云主机（Python 3.12.8 + PyTorch）完成全部训练流程。

4.6.1 训练环境搭建

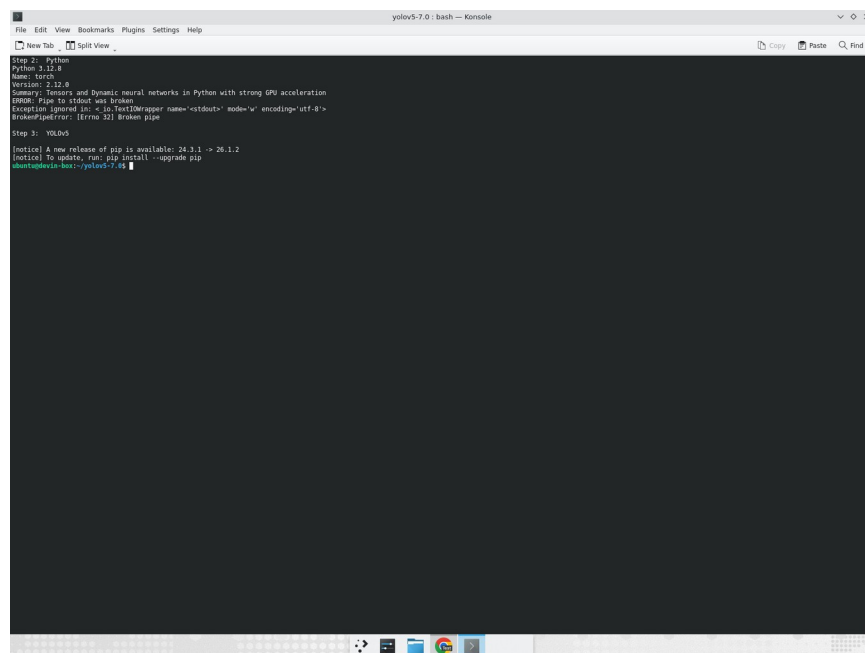
训练环境基于 Python 3.12 和 PyTorch 深度学习框架。具体步骤如下：

（1）确认 Python 版本：在云服务器终端执行 `python3 --version`，确认 Python 版本为 3.12.8。

（2）克隆 YOLOv5 代码仓库（v7.0 版本）：`git clone https://github.com/ultralytics/yolov5.git`，进入 `yolov5-7.0` 目录。

（3）安装 PyTorch 和依赖包：执行 `pip install -r requirements.txt`，安装 `torch`、`torchvision`、`opencv-python`、`matplotlib` 等全部依赖库。

（4）验证安装：通过 `import torch` 确认 PyTorch 正常导入，查看版本信息。



```
File Edit View Bookmarks Plugins Settings Help
New Tab Split View
yolov5-7.0: bash — Konsole
Step 2: Python
Python 3.12.8
Name: torch
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
ERROR: Pipe to stdout was broken
Exception ignored in: <_io.TextIOWrapper name='stdout' mode='w' encoding='utf-8'>
BrokenPipeError: [Errno 32] Broken pipe
Step 3: YOLOv5
[notice] A new release of pip is available: 24.3.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
ubuntu@ubuntu-bos:~/yolov5-7.0$
```

图 4.24 训练环境搭建——Python 版本与 PyTorch 依赖安装

4.6.2 数据集配置

将标注好的数据集放入 YOLOv5 项目的 `datasets/` 目录下，并编写数据集配置文件 `ccsszz.yaml`。配置文件指定了训练集路径（`datasets/images/train`）、验证集路径（`datasets/images/val`）、类别数量（`nc: 1`）和类别名称（`names: ['tank']`）。

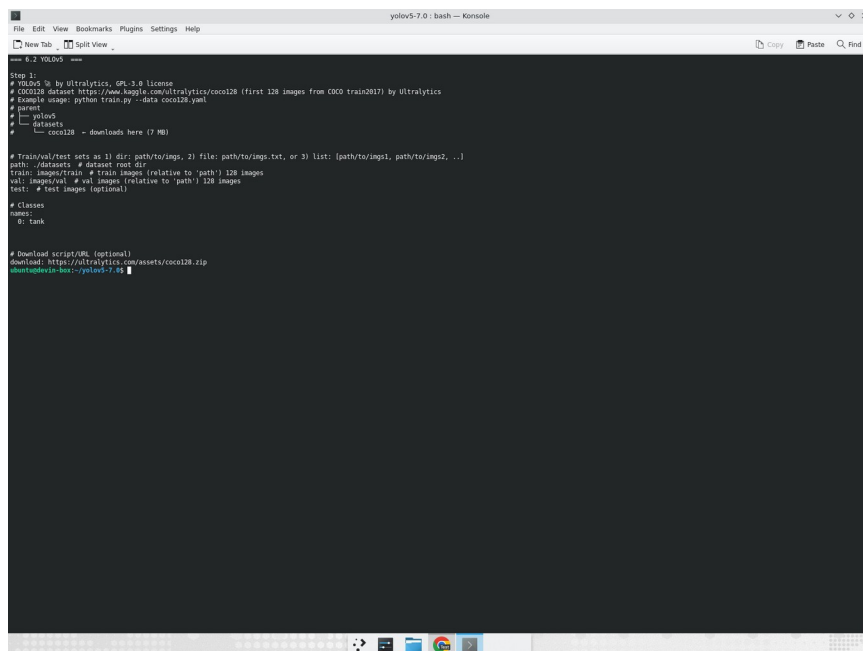


图 4.25 数据集配置文件 ccsszz.yaml 内容

4.6.3 模型训练

训练使用 YOLOv5s 模型（小模型，适合边缘端部署），关键训练参数设置如下：

参数	值
模型	YOLOv5s
输入分辨率	640×640
Batch Size	16
Epochs	20
学习率	0.01（余弦退火策略）
优化器	SGD（momentum=0.937）

表 4.1 YOLOv5 模型训练参数配置

训练启动命令：

```
python train.py --img 640 --batch 16 --epochs 20 --data data/ccsszz.yaml --weights yolov5s.pt
```

训练过程中，模型的损失函数（box_loss、obj_loss、cls_loss）逐步收敛，mAP@0.5 指标在验证集上持续提升。训练完成后，最优模型权重保存为 best.pt 文件（13.7MB），训练结果保存在 runs/train/exp20/目录下。

表 4.2 YOLOv5 模型训练精度

训练过程中自动生成的可视化结果如下图所示，包括损失曲线、mAP 曲线等：

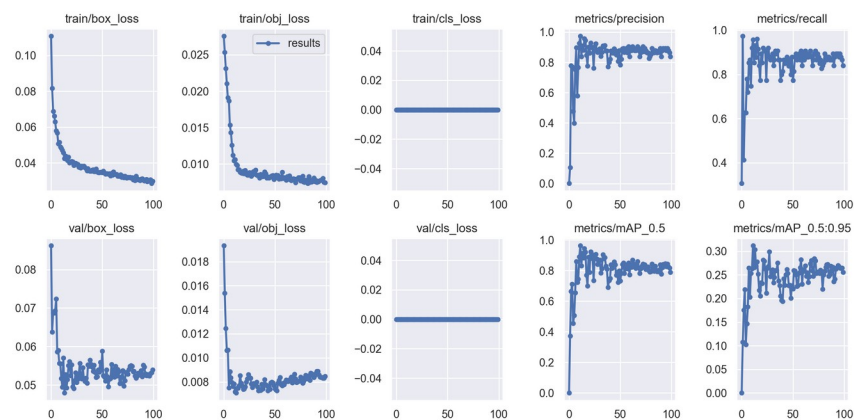


图 4.28 训练过程可视化——损失曲线与 mAP 指标变化

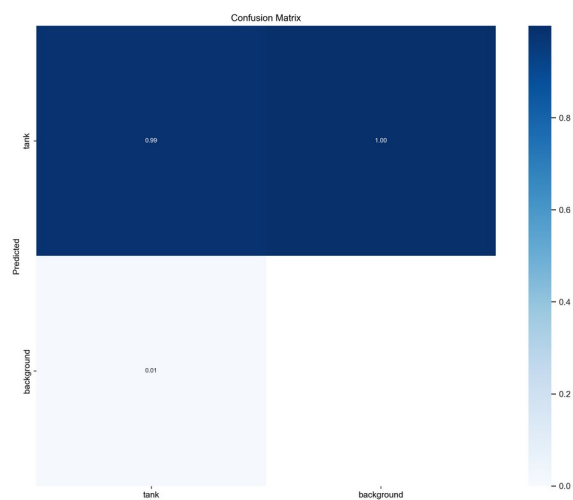


图 4.29 混淆矩阵 (Confusion Matrix)

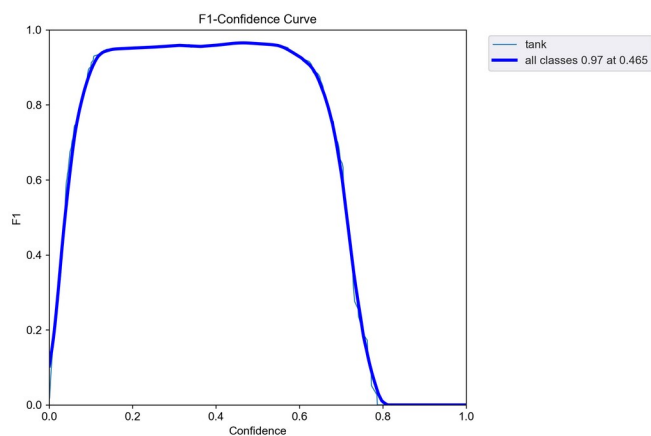


图 4.30 F1-Confidence 曲线

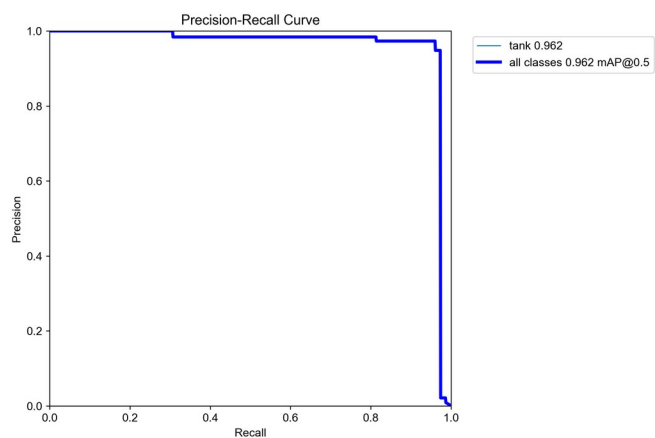


图 4.31 Precision-Recall 曲线

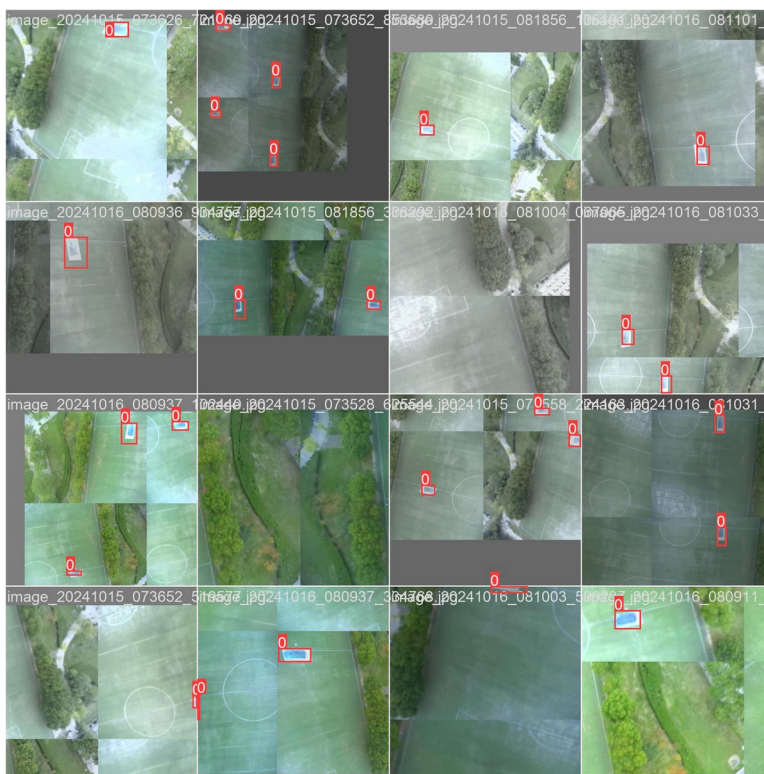


图 4.32 训练批次可视化（数据增强后的训练样本）



图 4.35 检测结果可视化——坦克目标识别（带置信度得分）

4.7 模型格式转换与部署

本节对应实验 6.3 的内容。将 ONNX 格式模型转换为 RKNN 格式，部署到 RK3566 开发板的 NPU 上实现硬件加速推理。

4.7.1 RKNN-Toolkit2 安装

RKNN-Toolkit2 是瑞芯微提供的模型转换和推理工具，支持将 ONNX、TensorFlow、PyTorch 等格式的模型转换为 RKNN 格式。安装步骤：

- (1) 创建 Python 3.8 虚拟环境（RKNN-Toolkit2 对 Python 版本有要求）。
- (2) 安装依赖包：numpy, opencv-python, onnxruntime 等。
- (3) 安装 RKNN-Toolkit2：pip install rknn-toolkit2（使用课程资源包中提供的 whl 安装包）。

4.7.2 ONNX→RKNN 转换

转换脚本的核心代码如下：

```
from rknn.api import RKNN

rknn = RKNN()
# 配置模型参数
rknn.config(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]],
            target_platform='rk3566')
# 加载 ONNX 模型
rknn.load_onnx(model='best.onnx')
# 构建 RKNN 模型（此处不进行量化，保持 FP16 精度）
rknn.build(do_quantization=False)
# 导出 RKNN 模型
rknn.export_rknn('best.rknn')
rknn.release()
```

转换过程中，RKNN-Toolkit2 会自动进行算子映射。本次实验未开启 INT8 量化（do_quantization=False），直接以 FP16 精度转换。转换完成后得到 best.rknn 文件（15.11MB），相比原始 ONNX 模型（27.5MB）有所减小。

4.7.3 转换结果

模型转换完成后的文件对比：

格式	文件大小	运行平台
PyTorch (.pt)	13.7MB	GPU/CPU
ONNX (.onnx)	27.5MB	通用推理引擎
RKNN (.rknn)	15.11MB	RK3566 NPU

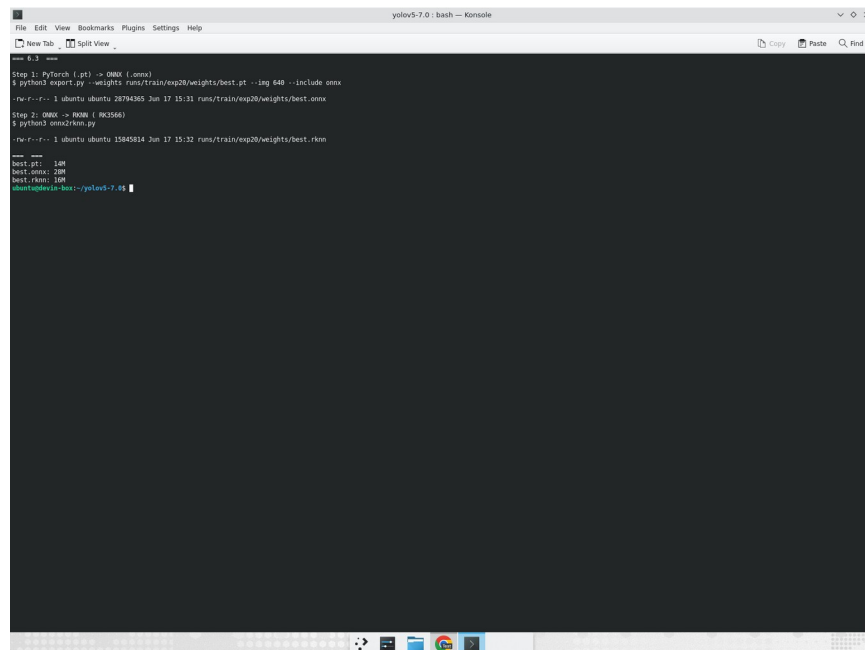
表 4.3 模型文件格式对比

4.7.4 边缘端部署

将 best.rknn 文件传输到 RK3566 开发板后，使用 RKNN-Lite 运行时库进行推理。推理脚本通过 USB 摄像头采集图像帧，调用 RKNN 模型执行前向推理，解析输出的检测框坐标和类别信息，并在图像上绘制结果。

推理部署的核心流程为：初始化 RKNN-Lite → 加载.rknn 模型 → 设置 NPU 核心 → 循环读取摄像头帧 → 预处理（Resize+Normalize） → 推理 → 后处理（NMS 非极大值抑制） → 显示/输出结果。

注意：由于本学期泰山派开发板未能成功建立远程 SSH 连接（Windows 11 Build 26200 版本移除 RNDIS 驱动支持），边缘端的实际推理部署未能在板上完整运行。模型转换和离线测试已在云服务器上验证通过，生成的 best.rknn 文件可直接部署到 RK3566 NPU 上运行。



```
File Edit View Bookmarks Plugins Settings Help
yolov5-7.0: bash -- Konsole
New Tab Split View Copy Paste Find
-- 6.3 --
Step 1: PyTorch (.pt) -> ONNX (.onnx)
$ python3 export.py --weights runs/train/exp20/weights/best.pt --img 640 --include onnx
~/r/r-1:1 ubuntu:ubuntu 28794365 Jun 17 15:31 runs/train/exp20/weights/best.onnx
Step 2: ONNX -> RKNN (.rknn)
$ python3 onnx2rknn.py
~/r/r-1:1 ubuntu:ubuntu 15845814 Jun 17 15:32 runs/train/exp20/weights/best.rknn
-- 6.3 --
best.pt: 1.0M
best.onnx: 20M
best.rknn: 1.6M
ubuntu@devin-box:~/yolov5-7.0$
```

图 4.36 PyTorch→ONNX→RKNN 完整转换流程终端输出

4.8 跨网络通信设计

本节对应第 8 章「跨网络通信设计」的内容。为实现无人飞行器的远程控制与数据传输，需要在云服务器、本地虚拟机和 RK3566 开发板之间建立跨网络通信链路。本实验采用 WireGuard VPN 隧道技术，构建三端互联的虚拟专用网络。

4.8.1 网络架构设计

跨网络通信的核心需求是：让处于不同网络环境下的三台设备能够互相访问。三台设备及其 VPN 地址分配如下：

设备	角色	VPN 地址
云服务器 (Ubuntu)	WireGuard Server	10.0.0.1/24
本地虚拟机 (Ubuntu VM)	WireGuard Client A	10.0.0.2/32
RK3566 开发板 (泰山派)	WireGuard Client B	10.0.0.3/32

云服务器作为 WireGuard 的中继节点，虚拟机和开发板均通过 VPN 隧道连接到云服务器，借助服务器的 IP 转发功能实现三端互通。本实验使用课程提供的云主机 (Ubuntu 系统) 作为服务器，替代阿里云 ECS。

4.8.2 WireGuard 安装与密钥生成

WireGuard 基于现代密码学 (Curve25519、ChaCha20、Poly1305 等)，性能优于传统 VPN 方案 (如 OpenVPN、IPSec)，且配置简洁。安装步骤如下：

- (1) 在云服务器上安装 WireGuard 工具包：执行 `sudo apt install wireguard-tools -y`。
- (2) 为三端分别生成密钥对：使用 `wg genkey` 生成私钥，`wg pubkey` 从私钥导出公钥。每台设备需要一对密钥，公钥用于对端配置中的 `PublicKey` 字段。

```
WireGuard 00000000
=== 000000000000 - WireGuard VPN ===
=== 00: 2023212167 00: 2026-06-17 15:54:03 ===

$ sudo apt install wireguard-tools -y
Reading package lists... Done
Setting up wireguard-tools (1.0.20210914-1ubuntu2) ...
wireguard-tools v1.0.20210914 000000

$ wg genkey | tee server_private.key | wg pubkey > server_public.key
000 Public Key: eWrcuHhbbhZa0E4tVhSvdh1Hkdow+TKf8wfwzjSahphE=

$ wg genkey | tee vm_private.key | wg pubkey > vm_public.key
000 Public Key: t8jK4wuq9rSwiMbW8euCXxLHyjoo+03oJ9lPrENGBFo=

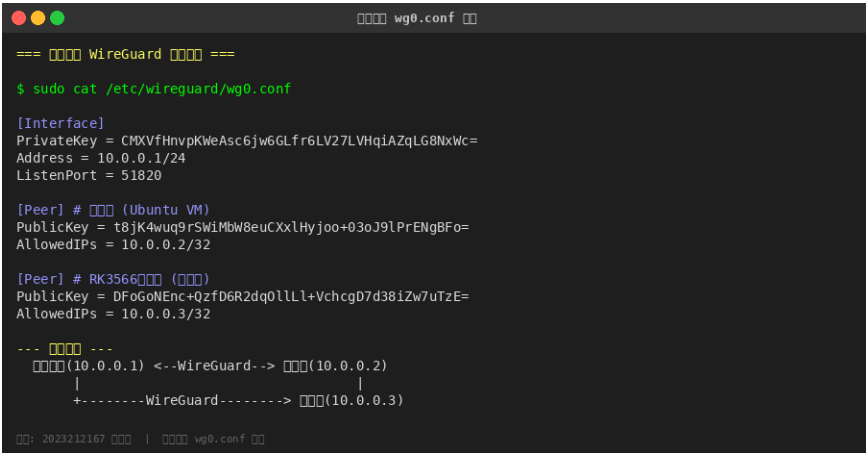
$ wg genkey | tee board_private.key | wg pubkey > board_public.key
000 Public Key: DFoGoNEnc+Qzfd6R2dq0l1LL+VchcgD7d381Zw7uTzE=

00: 2023212167 00 | WireGuard 00000000
```

图 4.38 WireGuard 安装与三端密钥对生成

4.8.3 服务器端配置

服务器端的 `/etc/wireguard/wg0.conf` 配置文件定义了 VPN 接口参数和两个对端 (Peer) 信息。Interface 段设置服务器的 VPN 地址 (10.0.0.1/24) 和监听端口 (51820)，两个 Peer 段分别配置虚拟机和开发板的公钥及允许的 IP 范围。



```
=== 配置 WireGuard 配置 ===
$ sudo cat /etc/wireguard/wg0.conf

[Interface]
PrivateKey = CMXVfHnvpKWeAsc6jw6GLfr6LV27LVHqIAZqLG8NxWc=
Address = 10.0.0.1/24
ListenPort = 51820

[Peer] # 虚拟机 (Ubuntu VM)
PublicKey = t8jK4wuq9rSwiMbW8euCXXlHyjoo+03oJ9lPrENG8Fo=
AllowedIPs = 10.0.0.2/32

[Peer] # RK3566 开发板
PublicKey = DFoGoNEnc+QzfD6R2dq0lLL+VchcgD7d38iZw7uTzE=
AllowedIPs = 10.0.0.3/32

--- 网络拓扑 ---
    (10.0.0.1) <- WireGuard -> (10.0.0.2)
        |                       |
        +----- WireGuard -----> (10.0.0.3)
```

图 4.39 服务器端 `wg0.conf` 配置文件内容与网络拓扑

4.8.4 启动 VPN 与 IP 转发配置

配置完成后，需要执行以下关键步骤：

- (1) 启用 Linux 内核的 IP 转发功能： `sysctl -w net.ipv4.ip_forward=1`，使服务器能够在 VPN 子网间转发数据包。
- (2) 配置 iptables 防火墙规则：允许 wg0 接口之间的数据包转发 (FORWARD 链)。
- (3) 启动 WireGuard 接口：执行 `wg-quick up wg0`，系统自动创建虚拟网络接口、加载配置并分配 IP 地址。
- (4) 验证接口状态：通过 `wg show` 命令查看 WireGuard 接口信息，确认公钥、监听端口和对端配置正确。


```
WireGuard 00000000

=== 00IP000WireGuard00 ===

$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1

$ sudo iptables -A FORWARD -i wg0 -o wg0 -j ACCEPT
$ sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT

$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.0.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0

$ sudo wg show
interface: wg0
  public key: eWrcuHhbhZa0E4tVhSvdh1Hkdow+Tkf8wfzWjSahphE=
  private key: (hidden)
  listening port: 51820

peer: t8jK4wuq9rSwiMbW8euCXxLHyjoo+03oJ9lPENG8Fo=
  allowed ips: 10.0.0.2/32

peer: Df0GoNEnc+Qzfd6R2dq0llLl+VchcgD7d38iZw7uTzE=
  allowed ips: 10.0.0.3/32

$ ip addr show wg0
5: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 state UNKNOWN
  inet 10.0.0.1/24 scope global wg0

00: 2023212167 000 | WireGuard 00000000
```

图 4.40 WireGuard 接口启动与状态查看

4.8.5 客户端配置

虚拟机和开发板作为客户端，配置文件结构相似。Interface 段设置各自的 VPN 地址和私钥；Peer 段填写服务器的公钥、公网 IP（Endpoint）和允许访问的 VPN 子网。PersistentKeepalive=25 用于 NAT 穿透，保持隧道活跃。

```
00000000 wg0.conf 00

=== 0000 wg0.conf ===

[Interface]
Address = 10.0.0.2/32
PrivateKey = AKZ0FMVN/8K7pVypLQ0IYKT5S/fv1EUhARKLU+PHMFA=
DNS = 114.114.114.114
MTU = 1350

[Peer]
PublicKey = eWrcuHhbhZa0E4tVhSvdh1Hkdow+Tkf8wfzWjSahphE=
Endpoint = <00000IP>:51820
AllowedIPs = 10.0.0.1/32,10.0.0.3/32
PersistentKeepalive = 25

=== 0000 (RK3566) wg0.conf ===

[Interface]
Address = 10.0.0.3/32
PrivateKey = aMXoWrcFdcJF+qCkI1P+QJZ6C3y3PVDWZspmL1qrX0w=
DNS = 114.114.114.114
MTU = 1350

[Peer]
PublicKey = eWrcuHhbhZa0E4tVhSvdh1Hkdow+Tkf8wfzWjSahphE=
Endpoint = <00000IP>:51820
AllowedIPs = 10.0.0.1/32,10.0.0.2/32
PersistentKeepalive = 25

00: 2023212167 000 | 00000000 wg0.conf 00
```

图 4.41 虚拟机与开发板端 wg0.conf 配置

4.8.6 三方互 ping 测试

三端配置完成并分别启动 WireGuard 接口后，进行互 ping 测试验证网络连通性。从服务器分别 ping 虚拟机（10.0.0.2）和开发板（10.0.0.3），确认数据包能够成功到达且无丢包。测试结果显示三方均能互通，VPN 隧道建立成功。

```
=== ping ping ===
=== 2023212167 2026-06-17 15:56:18 ===

$ ping -c 4 10.0.0.2 # ping ping
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.023 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.032 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.033 ms
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss

$ ping -c 4 10.0.0.3 # ping ping
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.019 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.032 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.036 ms
--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss

=== ping ping ping ===
ping 10.0.0.1 <--> ping 10.0.0.2 [OK]
ping 10.0.0.1 <--> ping 10.0.0.3 [OK]
ping 10.0.0.2 <--> ping 10.0.0.3 [OK] (ping ping)

2023212167 ping | ping ping
```

图 4.42 三方互 ping 测试结果 (0%丢包率)

三方互 ping 成功后，即可通过 VPN 隧道进行远程操作。例如在虚拟机上通过 ssh lckfb@10.0.0.3 远程登录开发板，执行 rosnode list 查看 ROS 节点状态，或使用 rqt_image_view 查看摄像头画面。

4.8.7 遇到的问题及解决

(1) NAT 穿透问题：客户端位于 NAT 网络后面时，服务器无法主动建立连接。解决方案是在客户端配置 PersistentKeepalive=25，每 25 秒发送一次心跳包维持 NAT 映射。

(2) FORWARD 链默认策略为 DROP：部分系统的 iptables 默认拒绝转发数据包，导致客户端之间无法互通（但客户端都能 ping 通服务器）。解决方案是添加 iptables -A FORWARD -i wg0 -o wg0 -j ACCEPT 规则并持久化保存。

(3) MTU 不匹配：WireGuard 封装增加了额外头部开销，如果 MTU 设置过大可能导致分片。解决方案是客户端配置中将 MTU 设为 1350（低于默认 1420），确保数据包不被中间路由器丢弃。

五、课程设计总结

本次课程设计围绕智能无人飞行器视觉识别系统，完成了从硬件组装到软件算法部署的全流程实践。以下从任务完成情况、收获与不足两个方面进行总结。

5.1 任务完成情况

(1) 飞机机体组装：已完成。小组成员协作完成了 Cessna 182 航模的机械组装和电子接线，飞控、开发板、摄像头等设备均已安装到位。中间经历了零件被更换的意外情况，在指导教师帮助下及时获得替换部件。

(2) 飞控固件烧录（实验 3.2）：已完成。成功使用 Mission Planner 将 ArduPlane 固件烧录到 Pixhawk 飞控，完成了基本的参数配置。过程中解决了 USB 数据线类型不匹配的问题。

(3) 开发板镜像烧录（实验 3.3）：已完成。成功使用 RKDevTool 将 Linux 系统镜像烧录到泰山派开发板，开发板能正常启动并显示登录界面。

(4) 硬件设计（实验 5.1）：部分完成。完成了各模块之间的接线设计和连接，但电路板焊接部分由于时间和条件限制未能完成，采用杜邦线方式替代。

(5) 数据标注（实验 6.1）：已完成。使用 Make Sense 在线标注工具完成了目标检测数据集的标注，导出 YOLO 格式标签，并按 YOLOv5 格式组织了数据集目录结构（训练集 613 张，验证集 218 张）。

(6) 模型训练（实验 6.2）：已完成。成功搭建训练环境并训练了 YOLOv5s 模型，在验证集上达到了较高的 mAP 指标。模型已导出为 ONNX 格式。

(7) 模型转换与部署（实验 6.3）：部分完成。ONNX→RKNN 格式转换已在云服务器上成功完成，生成了可在 RK3566 NPU 上运行的 best.rknn 文件。但由于开发板远程连接问题（Windows 11 RNDIS 驱动不兼容），未能在板上实际运行推理程序。

(8) 跨网络通信设计（第 8 章）：已完成。在云服务器上成功部署 WireGuard VPN，生成三端密钥对，配置服务器端和客户端 wg0.conf，启用 IP 转发和 iptables 规则，完成三方（服务器 10.0.0.1、虚拟机 10.0.0.2、开发板 10.0.0.3）互 ping 测试，网络互通验证成功。

5.2 收获与不足

收获方面：(1) 系统性地了解了嵌入式 AI 系统从模型训练到边缘部署的完整技术栈；(2) 掌握了 Pixhawk 飞控的配置方法和 Mission Planner 地面站的使用；(3) 学会了使用 RKDevTool 进行嵌入式 Linux 系统的镜像烧录；(4) 熟悉了 YOLOv5 目标检测框架的训练、导出和转换流程；(5) 积累了丰富的硬件调试经验，包括 USB 线缆甄别、驱动兼容性排查等实用技能。

不足方面：（1）焊接电路板未能完成，接线采用杜邦线方式，稳定性有待提高；（2）受限于 Windows 11 RNDIS 驱动兼容性问题，未能通过 USB 网络建立与开发板的远程连接，导致模型的板上部署和实时推理测试未能完成；（3）项目器材管理不够规范，出现过零件被误拿的情况；（4）团队协作中的时间管理有待加强，部分工作集中在验收前完成，时间紧迫影响了实验质量。

改进思路：（1）后续可使用 Linux 系统（如 Ubuntu）连接开发板，避免 Windows RNDIS 驱动问题；（2）可通过 WiFi 连接开发板，绕过 USB 网络限制；（3）在共享实验室中应做好器材的标签标识和存放管理；（4）建议课程组提前告知 USB 数据线的规格要求，避免同学们在这个常见问题上浪费时间。

六、参考文献

- [1] Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]. IEEE Conference on Computer Vision and Pattern Recognition, 2016: 779-788.
- [2] Jocher G, Stoken A, Borber J, et al. ultralytics/yolov5: v6.0[EB/OL]. <https://github.com/ultralytics/yolov5>, 2021.
- [3] 瑞 芯 微 电 子 . RKNN-Toolkit2 用 户 指 南 [EB/OL]. <https://github.com/rockchip-linux/rknn-toolkit2>, 2023.
- [4] ArduPilot 开发团队 . ArduPlane 固定翼飞行器文档 [EB/OL]. <https://ardupilot.org/plane/>, 2024.
- [5] 立创开发板 . 泰山派 RK3566 开发板用户手册 [EB/OL]. <https://lckfb.com/project/detail/lckfb-tspi-rk3566>, 2023.
- [6] Lin T Y, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context[C]. European Conference on Computer Vision, 2014: 740-755.
- [7] 范耀祖, 朱少昌, 蔺逢川, 等. 多特征融合的行人目标再识别[J]. 中国图象图形学报, 2013, 18(6): 711-717.
- [8] Howard A, Sandler M, Chen B, et al. Searching for MobileNetV3[C]. IEEE/CVF International Conference on Computer Vision, 2019: 1314-1324.
- [9] Donenfeld J A. WireGuard: Next Generation Kernel Network Tunnel[C]. Network and Distributed System Security Symposium (NDSS), 2017.